دانشگاه صنعتی کرمانشاه

پیشنهاد تفصیلی پروژهٔ درس مهندسی نرم‌افزار

فرنما (Farnama)

زهرا میرزائی	امیرمهدی عزیزی	امیرحسین عالیان
۴۰۲۱۱۲۰۰۳۴	۴۰۲۱۱۲۰۰۱۹	۴۰۲۱۱۲۰۰۱۷

استاد درس: دکتر سودابه محمدی

نسخه ۱.۱ سند - آبان ۱۴۰۵

فهرست مطالب

۱ مقدمه و هدف پروژه ۳

۲ آماده سازی محیط کار تیم ۵
۱.۲ ایجاد مخزن‌ها ۵
۲.۲ توضیح مخزن‌ها ۵

۳ اطلاعات تیم و اقلام پروژه ۶
۱.۳ معرفی اعضای تیم ۶
۲.۳ ابزارهای ارتباطی تیم ۷

۴ توصیف محصول نرم‌افزاری ۸
۱.۴ معرفی ایده و هدف محصول ۸
۲.۴ قابلیت‌های اصلی محصول ۹
۱.۲.۴ مدیریت انجمن‌ها و شوراها ۹

۲.۲.۴ ثبت و پیگیری درخواست برگزاری دوره و رویداد ۹
۳.۲.۴ برگزاری انتخابات ۹
۴.۲.۴ اطلاع‌رسانی و اعلان‌ها ۱۰
۵.۲.۴ سطوح دسترسی ۱۰
۳.۴ قابلیت‌های توسعه‌ای یا اهداف تکمیلی ۱۰

۵ نیازمندی‌های غیرکارکردی ۱۱

۶ الزامات عمومی و بیرونی محصول ۱۲
۱.۶ مدیریت خطا ۱۲
۲.۶ امکان نصب و اجرا ۱۲
۳.۶ قابلیت ساخت از کد منبع ۱۲
۴.۶ تناسب محدودهٔ پروژه با منابع تیم ۱۲

۷ فرایند کار تیم و برنامهٔ توسعه ۱۳
۱.۷ انتخاب ابزارها و فناوری‌ها ۱۳
۱.۱.۷ گزینه‌های بررسی‌شده برای تصمیم‌های اصلی ۱۴
۲.۱.۷ معماری سامانه ۱۵
۲.۷ تعریف و توجیه نقش اعضای تیم ۱۵
۳.۷ برنامهٔ زمان‌بندی و نقاط عطف ۱۶
۴.۷ شناسایی و تحلیل ریسک‌ها ۱۷
۱.۴.۷ ناآشنایی با فناوری‌های اصلی ۱۷
۲.۴.۷ محدودیت دسترسی از ایران به سرویس‌های خارجی ۱۷
۳.۴.۷ کمبود زمان ۱۷
۴.۴.۷ ناسازگاری اپ و سرور ۱۸
۵.۷ برنامهٔ دریافت بازخورد ۱۸

۱ مقدمه و هدف پروژه

۱. محصول نرم‌افزاری ما چیست و چه هدفی را دنبال می‌کند؟

فرنما سامانه‌ای برای معاونت فرهنگی دانشگاه است که از یک اپلیکیشن اندروید و یک سرور تشکیل شده است. هدف آن این است که ثبت و پیگیری درخواست برگزاری دوره، مدیریت انجمن‌ها و شوراها، برگزاری انتخابات و اطلاع‌رسانی برنامه‌ها، به‌جای روش حضوری و کاغذی و کانال‌های پراکنده، در یک‌جا و به‌صورت غیرحضوری انجام شود.

۲. چه کاربران یا ذی‌نفعانی از محصول استفاده خواهند کرد؟

کاربران هدف اصلی و اولیهٔ فرنما دبیران و اعضای تشکل‌های دانشجویی (انجمن‌های علمی، شورای صنفی و دیگر تشکل‌ها) و کارشناسان معاونت فرهنگی هستند؛ زیرا هستهٔ محصول، ثبت و پیگیری درخواست‌های اداری میان این دو گروه است. دانشجویان عمومی دانشگاه در اولویت بعدی قرار دارند و امکانات ویژهٔ آن‌ها جزو اهداف توسعه‌ای پروژه است. ذی‌نفع سازمانی، خود معاونت فرهنگی است که فرایندهایش ساده‌تر و شفاف‌تر می‌شود.

۳. محصول چه قابلیت‌هایی ارائه می‌دهد؟

- **مدیریت تشکل‌ها:** هر انجمن یا شورا صفحهٔ مخصوص خود را شامل فهرست اعضا و سابقهٔ فعالیت‌ها دارد که دبیر آن را به‌روز نگه می‌دارد.
- **ثبت و پیگیری درخواست:** دبیر درخواست برگزاری دوره، کارگاه یا رویداد را ثبت می‌کند، کارشناس آن را تأیید یا رد می‌کند و وضعیت درخواست در هر مرحله برای دبیر قابل مشاهده است.
- **برگزاری انتخابات:** کارشناس انتخابات تشکل‌ها را تعریف می‌کند و از ثبت‌نام نامزدها تا اعلام نتیجه بر آن نظارت دارد. سامانه تضمین می‌کند که هر نفر به تعداد مجاز رأی بدهد تا همه آرا معتبر باشند و هیچ رأی باطله‌ای وجود نداشته باشد.
- **اعلان‌ها:** اطلاع از تغییر وضعیت درخواست‌ها، رویدادهای جدید و زمان‌بندی انتخابات.
- **سطوح دسترسی:** نقش‌های دانشجو، عضو تشکل، دبیر تشکل و کارشناس از یکدیگر جدا هستند و سرور در هر درخواست، نقش کاربر را بررسی می‌کند.

انتخابات تنها قابلیتی است که همهٔ دانشجویان را در بر می‌گیرد. سایر امکانات ویژهٔ دانشجویان، مانند ثبت‌نام در دوره‌ها و نظرسنجی، جزو اهداف توسعه‌ای است.

۴. نیازمندی‌های کارکردی و غیرکارکردی سیستم چیست؟

نیازمندی‌های کارکردی همان قابلیت‌های بند ۳ هستند. نیازمندی‌های غیرکارکردی اصلی عبارت‌اند از:

- **امنیت و حریم خصوصی:** احراز هویت امن و مخفی ماندن رأی‌ها. ممکن است برخی کاربران به دلایل امنیتی یا حریم خصوصی نخواهند اپلیکیشن را نصب کنند؛ برای آن‌ها یک صفحهٔ وب متصل به همان API در نظر گرفته می‌شود، به‌طوری که قواعد امنیتی در هر دو مسیر یکسان اعمال شود.
- **قابلیت استفاده:** رابط فارسی و راست‌به‌چپ؛ همچنین نیاز به ورود دوباره تا حد ممکن کاهش می‌یابد و این کار با احراز هویت مبتنی بر توکن انجام می‌شود، زیرا ورود در هر

بار استفاده، به‌ویژه در وب، آزاردهنده است.

- **یکپارچگی داده:** هر نفر در هر انتخابات به تعداد مشخصی رأی دارد و این قاعده در خود پایگاه داده تضمین می‌شود.
- **نگهداشت‌پذیری:** کل سرور با یک دستور و در محیطی یکسان روی سیستم همهٔ اعضا اجرا می‌شود (با Docker Compose).

۵. **هر یک از اعضای تیم چه مسئولیت‌هایی بر عهده دارند؟**

زهرا میرزائی مسئول توسعهٔ اپلیکیشن اندروید، امیرمهدی عزیزی مسئول توسعهٔ سرور و پایگاه داده، و امیرحسین عالیان مسئول نقش ترکیبی (Hybrid)، یعنی حفظ یکپارچگی اپ و سرور و زیرساخت مشترک پروژه است.

۶. **پروژه چگونه، با چه ابزارهایی و بر اساس چه برنامه‌ای توسعه داده خواهد شد؟**

اپلیکیشن اندروید با Kotlin و Room و سرور با Python، Django و PostgreSQL توسعه داده می‌شود و سرور با Docker Compose اجرا می‌شود. کد و وظایف روی GitHub مدیریت می‌شوند و مستندات با LaTeX نوشته می‌شوند. توسعه به‌صورت هفتگی و افزایشی پیش می‌رود و در هر هفته یک قابلیت به‌طور کامل در سرور و اپ پیاده‌سازی می‌شود.

۷. **چه ریسک‌هایی ممکن است مانع تکمیل موفقیت‌آمیز پروژه شوند؟**

ناآشنایی تیم با برخی فناوری‌های اصلی، محدودیت دسترسی از ایران به سرویس‌ها و مخازن خارجی (Docker، Gradle و اعلان‌های گوگل)، کمبود زمان، و ناسازگاری اپ و سرور که هم‌زمان توسعه داده می‌شوند.

۲ آماده سازی محیط کار تیم

۱.۲ ایجاد مخزن‌ها

کد و مستندات پروژه در سه مخزن نگه‌داری می‌شوند. **مخزن** `farnama-docs`**، مخزن اصلی پروژه است** و مستندات و پیوند به دو مخزن دیگر در آن قرار دارد.

جدول ۱: مخزن‌های پروژه

مخزن	محتوا	نشانی
`farnama-docs`	مستندات (**مخزن اصلی**)	`github.com/amirh4liyan/farnama-docs`
`farnama-android`	کد اپلیکیشن اندروید	`github.com/amirh4liyan/farnama-android`
`farnama-backend`	کد سمت سرور	`github.com/amirh4liyan/farnama-backend`

> **توجه:** ممکن است ساختار پروژه به یک مخزن تغییر کند. در آن صورت، محتویات مخزن‌ها با یک‌دیگر ترکیب شده و هر سه در یک مخزن نگه‌داری خواهند شد.

سایر اقلام پروژه، شامل نمونه‌های اولیهٔ رابط کاربری، مستندات طراحی و سامانهٔ مدیریت وظایف، هنوز آماده نشده‌اند و نشانی هر کدام پس از آماده شدن به این بخش اضافه می‌شود.

۲.۲ توضیح مخزن‌ها

کد و مستندات پروژه در سه مخزن جداگانه نگه‌داری می‌شوند: `farnama-android` برای کد اپلیکیشن، `farnama-backend` برای کد سمت سرور، و `farnama-docs` برای مستندات، که مخزن اصلی پروژه است و پیوند به دو مخزن دیگر در آن قرار دارد. دلیل این تفکیک آن است که اپلیکیشن و سرور ابزارهای ساخت، وابستگی‌ها و چرخهٔ انتشار کاملاً متفاوتی دارند (Gradle و Android Studio در برابر Python و Docker)؛ با جدا بودن مخزن‌ها، هر عضو در محدودهٔ تمرکز خود کار می‌کند، تعارض‌های ادغام کد کاهش می‌یابد و هر بخش مستقل از دیگری نسخه‌گذاری و منتشر می‌شود. ارتباط میان دو مخزن کد نیز فقط از طریق قرارداد API برقرار است، که همین مرز را در سطح مخازن هم صریح می‌کند.

تفکیک مخازن هزینهٔ هماهنگی میان آن‌ها را افزایش می‌دهد؛ برای کنترل این هزینه، سه سازوکار در نظر گرفته شده است. نخست، مستندات OpenAPI از کد سرور در مخزن `farnama-backend` تولید می‌شود و کلاینت اپلیکیشن از روی همان فایل ساخته می‌شود، به‌طوری که هر تغییر در API به خطای کامپایل در اپلیکیشن تبدیل می‌شود و پنهان نمی‌ماند. دوم، روند کار در هر سه مخزن یکسان است: شاخهٔ اصلی محافظت‌شده است و هر تغییر فقط از طریق Pull Request و با بازبینی دست‌کم یک عضو دیگر ادغام می‌شود. سوم، نسخه‌های قابل تحویل اپلیکیشن و سرور با برچسب نسخهٔ یکسان منتشر می‌شوند تا مشخص باشد کدام نسخهٔ اپلیکیشن با کدام نسخهٔ سرور سازگار است.

۳ اطلاعات تیم و اقلام پروژه

۱.۳ معرفی اعضای تیم

زهرا میرزائی

- **نقش اصلی:** توسعه‌دهندهٔ اپلیکیشن اندروید.
- **مسئولیت‌ها:** پیاده‌سازی صفحات اپ برای دبیران و اعضای تشکل‌ها و صفحهٔ رأی‌گیری، ذخیره‌سازی محلی با Room، و مدیریت ورود و اعلان‌ها در اپ.
- **مهارت‌ها:** آشنایی با Python، Git و لینوکس.

امیرمهدی عزیزی

- **نقش اصلی:** توسعه‌دهندهٔ سمت سرور.
- **مسئولیت‌ها:** طراحی پایگاه دادهٔ PostgreSQL، پیاده‌سازی API با Django، احراز هویت و کنترل دسترسی، و منطق درخواست‌ها و انتخابات.
- **مهارت‌ها:** آشنایی با Python، Git و لینوکس.

امیرحسین عالیان

- **نقش اصلی:** یکپارچه‌سازی (Hybrid).
- **مسئولیت‌ها:** حفظ یکپارچگی میان اپ و سرور، شامل قرارداد API، راه‌اندازی Docker Compose، پنل کارشناسان معاونت با Django Admin، صفحهٔ وب جایگزین، هماهنگی مستندات و رفع تعارض‌های ادغام کد (Merge Conflicts) در Git. این نقش در صورت نیاز در همهٔ بخش‌های پروژه مشارکت می‌کند.
- **مهارت‌ها:** تجربهٔ برنامه‌نویسی اندروید، کار با Python، Docker و Git، و تسلط قابل قبول بر لینوکس.

مسئولیت‌های فوق تمرکز اصلی هر عضو را نشان می‌دهند. در صورت نیاز، اعضا برای افزایش سرعت کار در بخش‌های دیگر نیز به یکدیگر کمک می‌کنند.

۲.۳ ابزارهای ارتباطی تیم

کانال‌های ارتباطی

تیم پس از آزمودن چند ابزار ارتباطی، ابزارهای زیر را به ترتیب اولویت انتخاب کرد:

۱. **تلگرام (اولویت اول):** هماهنگی روزمره در یک گروه تلگرامی انجام می‌شود و جلسات، هر زمان که دست‌کم دو نفر از اعضا بتوانند حاضر باشند، به‌صورت Video Chat در همین گروه برگزار می‌شوند. دلیل این انتخاب آن است که اعضا تلگرام را مرتب بررسی می‌کنند، تلگرام امکان ضبط تماس را به‌صورت داخلی دارد و کیفیت اشتراک صفحه در آن بهتر بود. اگر حضور هم‌زمان اعضا ممکن نباشد، جلسه به‌صورت غیرهم‌زمان (Asynchronous) و با پیام صوتی پیش می‌رود: هر عضو هر زمان که توانست پیام‌ها را می‌شنود و پاسخ می‌دهد.

۲. **دیسکورد (اولویت دوم):** یک سرور دیسکورد برای تماس تصویری ساخته شده است و اطلاعات پروژه در کانال‌های موضوعی و دسته‌بندی‌شدهٔ آن نگه‌داری می‌شود که مدیریتی تمیزتر از گروه تلگرامی دارد.

۳. **Google Meet (اولویت سوم):** در شرایط کیفیت پایین اینترنت از آن استفاده می‌شود.

واتس‌اپ و آلوکام نیز آزمایش شدند، اما کیفیت مناسبی نداشتند.

ثبت و پیگیری

وظایف اعضا، تصمیم‌های مهم و تغییرات اساسی پروژه در GitHub ثبت و پیگیری می‌شوند:

- **وظایف:** با GitHub Issues و GitHub Projects تعریف و پیگیری می‌شوند.
- **تغییرات کد:** از طریق Pull Request و با بازبینی دست‌کم یک عضو دیگر ادغام می‌شوند.
- **تصمیم‌های مهم:** هر تصمیم به‌صورت یک Issue با برچسب `decision` و همراه با دلیل آن ثبت می‌شود.

۴ توصیف محصول نرم‌افزاری

۱.۴ معرفی ایده و هدف محصول

۱. **مسئلهٔ اصلی**

ثبت و پیگیری درخواست‌های اداری تشکل‌های دانشجویی نزد معاونت فرهنگی، مانند درخواست برگزاری دوره و رویداد، و نیز برگزاری انتخابات تشکل‌ها به‌صورت حضوری و کاغذی انجام می‌شود.

۲. **اهمیت مسئله**

دبیران برای هر درخواست باید مراجعهٔ حضوری داشته باشند و زمان زیادی را صرف گرفتن مجوز کنند، و وضعیت درخواست نیز فقط با پیگیری شخصی معلوم می‌شود. این بار اداری هزینهٔ دیگری هم دارد: بر اساس بازخوردهایی که تیم از دانشجویان گرفته است، بسیاری از دانشجویان به‌دلیل دشواری کارهای اداری انجمن‌ها تمایلی به عضویت در آن‌ها ندارند و به این ترتیب، انجمن‌ها از حضور افرادی محروم می‌شوند که استعداد خوبی دارند و می‌توانند برای انجمن مفید باشند. علاوه بر این، بخش قابل‌توجهی از آموزش در سال‌های اخیر مجازی بوده و حضور در دانشگاه همیشه ممکن نیست.

۳. **کاربران هدف**

کاربران اصلی، دبیران و اعضای تشکل‌ها و کارشناسان معاونت فرهنگی هستند. دانشجویان عمومی کاربران ثانویه‌اند و در فاز اول فقط در انتخابات شرکت می‌کنند.

۴. **وضعیت فعلی**

درخواست‌ها با فرم کاغذی و مراجعهٔ حضوری ثبت می‌شوند و اطلاع‌رسانی برنامه‌ها در کانال‌های تلگرامی و به‌صورت پراکنده انجام می‌شود.

۵. **راه‌حل پیشنهادی**

یک اپلیکیشن اندروید برای دبیران، اعضای تشکل‌ها و دانشجویان، و یک پنل مدیریت وب برای کارشناسان که هر دو به یک سرور مشترک متصل‌اند. برای کاربرانی که نمی‌خواهند اپلیکیشن را نصب کنند، یک صفحهٔ وب متصل به همان API در نظر گرفته شده است.

۶. **ارزش اصلی محصول**

سرعت بخشیدن به فعالیت‌های انجمن‌ها و تشکل‌ها، از طریق ثبت و پیگیری غیرحضوری درخواست‌ها با وضعیت شفاف در هر مرحله و برگزاری انتخاباتی که اعتبار آراء را خود سامانه تضمین می‌کند. کاهش بار اداری، فعالیت در انجمن‌ها را برای دانشجویان مستعد نیز جذاب‌تر می‌کند.

۷. **محدودهٔ پروژه**

- **داخل محدوده:** مدیریت انجمن‌ها و شوراها، ثبت و پیگیری درخواست برگزاری دوره و رویداد، برگزاری انتخابات انجمن‌ها و شورای صنفی، اطلاع‌رسانی و اعلان‌ها، و سطوح دسترسی بر اساس نقش.

- **اهداف توسعه‌ای:** ثبت‌نام دانشجویان در دوره‌ها و رویدادها، نظرسنجی و بازخورد، و بررسی درخواست‌ها توسط کارشناسان از طریق اپلیکیشن اندروید (فهرست کامل در بخش ۳.۴).
- **خارج از محدوده:** نسخهٔ iOS، پرداخت آنلاین، اتصال به سامانه‌های آموزشی دانشگاه مانند گلستان، گفت‌وگوی متنی میان کاربران، و امور غیرفرهنگی مانند خوابگاه و تغذیه.

۲.۴ قابلیت‌های اصلی محصول

۱.۲.۴ مدیریت انجمن‌ها و شوراها

- **هدف:** داشتن یک صفحهٔ رسمی و به‌روز برای هر تشکل.
- **کاربر:** دبیر تشکل.
- **عملکرد سیستم:** هر انجمن علمی، شورای صنفی یا تشکل دیگر صفحه‌ای شامل معرفی، فهرست اعضا و سابقهٔ فعالیت‌ها دارد. دبیر اطلاعات صفحه را ویرایش و عضو اضافه یا حذف می‌کند.
- **نتیجه برای کاربر:** مدیریت منظم تشکل بدون نیاز به فهرست‌ها و پرونده‌های پراکنده.

۲.۲.۴ ثبت و پیگیری درخواست برگزاری دوره و رویداد

- **هدف:** حذف فرم کاغذی و مراجعهٔ حضوری.
- **کاربر:** دبیر تشکل (ثبت درخواست) و کارشناس معاونت (بررسی آن).
- **عملکرد سیستم:** دبیر درخواست برگزاری دوره، کارگاه یا رویداد را با مشخصات آن (عنوان، زمان، مکان و توضیحات) ثبت می‌کند. کارشناس درخواست را تأیید یا رد می‌کند و در صورت رد، دلیل آن را ثبت می‌کند. وضعیت درخواست («ثبت‌شده»، «در حال بررسی»، «تأییدشده» یا «ردشده») در هر لحظه برای دبیر قابل مشاهده است.
- **نتیجه برای کاربر:** دبیر بدون مراجعهٔ حضوری مجوز می‌گیرد و همیشه از وضعیت درخواست خود آگاه است.

۳.۲.۴ برگزاری انتخابات

- **هدف:** برگزاری انتخابات غیرحضوری و قابل اعتماد.
- **کاربر:** کارشناس معاونت (تعریف و نظارت)، و اعضا و دانشجویان (نامزد شدن و رأی دادن).
- **عملکرد سیستم:** کارشناس انتخابات یک تشکل را با بازهٔ ثبت‌نام نامزدها و بازهٔ رأی‌گیری تعریف می‌کند. سامانه تضمین می‌کند که هر نفر در هر انتخابات حداکثر به تعداد مجاز رأی ثبت کند و رأی هیچ فردی برای دیگران قابل مشاهده نباشد. نتایج پس از پایان رأی‌گیری اعلام می‌شوند.
- **نتیجه برای کاربر:** شرکت در انتخابات بدون حضور فیزیکی و با اطمینان از سلامت آن.

۴.۲.۴ اطلاع‌رسانی و اعلان‌ها

- **هدف:** جایگزینی کانال‌های پراکنده.
- **کاربر:** همهٔ کاربران.
- **عملکرد سیستم:** با تغییر وضعیت یک درخواست، به دبیر همان تشکل اعلان ارسال می‌شود. با تعریف یک انتخابات و نزدیک شدن زمان رأی‌گیری، به رأی‌دهندگان آن انتخابات، و با تأیید یک رویداد جدید، به اعضای تشکل برگزارکننده اعلان ارسال می‌شود.
- **نتیجه برای کاربر:** آگاهی به‌موقع از هر تغییر، بدون نیاز به دنبال کردن چند کانال مختلف.

۵.۲.۴ سطوح دسترسی

- **هدف:** امنیت و جداسازی وظایف.
- **کاربر:** همهٔ کاربران.
- **عملکرد سیستم:** چهار نقش تعریف می‌شود: دانشجو، عضو تشکل، دبیر تشکل و کارشناس معاونت. سرور در هر درخواست، نقش کاربر و ارتباط او با تشکل مربوط را بررسی می‌کند؛ برای نمونه، دبیر فقط درخواست‌های تشکل خود را می‌بیند.
- **نتیجه برای کاربر:** هر کاربر فقط به امکانات مربوط به خود دسترسی دارد.

۳.۴ قابلیت‌های توسعه‌ای یا اهداف تکمیلی

اهداف زیر فقط پس از تکمیل پنج قابلیت اصلی و به ترتیب اولویت پیاده‌سازی می‌شوند:

۱. **ثبت‌نام در دوره‌ها و رویدادها:** دانشجویان برنامه‌های تأییدشده را می‌بینند و از داخل اپلیکیشن ثبت‌نام می‌کنند؛ سامانه از ثبت‌نام بیش از ظرفیت جلوگیری می‌کند.

۲. **بررسی درخواست‌ها با اپلیکیشن اندروید:** کارشناسان معاونت علاوه بر پنل وب، از داخل اپلیکیشن نیز درخواست‌ها را بررسی و تأیید می‌کنند. تا آن زمان، این کار از طریق پنل Django Admin انجام می‌شود.

۳. **گواهی شرکت در دوره:** پس از پایان دوره، برای شرکت‌کنندگان گواهی PDF صادر می‌شود. این قابلیت انگیزهٔ مستقیمی برای دانشجویان ایجاد می‌کند و با هدف جذب دانشجویان به انجمن‌ها هم‌راستاست.

۴. **نظرسنجی و بازخورد:** دانشجویان دربارهٔ دوره‌ها، رویدادها و خود اپلیکیشن نظر ثبت می‌کنند.

۵. **تأیید رأی با اثر انگشت:** پیش از ثبت رأی، قفل گوشی یا اثر انگشت درخواست می‌شود تا کسی که به گوشی باز دسترسی دارد نتواند به‌جای صاحب گوشی رأی دهد.

۶. **تقویم رویدادها:** نمایش همهٔ رویدادهای تأییدشده روی یک تقویم شمسی.

۷. **گزارش آماری برای معاونت:** تعداد درخواست‌ها، رویدادها و میزان مشارکت هر تشکل در بازهٔ زمانی دلخواه.

۵ نیازمندی‌های غیرکارکردی

جدول ۲: نیازمندی‌های غیرکارکردی و معیار سنجش آن‌ها

حوزه	نیازمندی	معیار سنجش
امنیت	رمزهای عبور فقط به‌صورت هش‌شده ذخیره می‌شوند و همهٔ ارتباطات روی HTTPS انجام می‌شود.	دستور `manage.py check --deploy` هیچ هشداری گزارش نمی‌دهد.
امنیت	ورود مبتنی بر توکن است: یک توکن دسترسی کوتاه‌مدت و یک توکن تمدید که با هر بار استفاده عوض می‌شود.	عمر توکن دسترسی ۱۵ دقیقه و عمر توکن تمدید ۳۰ روز است؛ توکن تمدیدی که یک بار استفاده شده، دوباره پذیرفته نمی‌شود.
امنیت	حدس زدن رمز با تلاش‌های پیاپی ممکن نیست.	پس از ۵ تلاش ناموفق، ورود به آن حساب برای ۱۵ دقیقه مسدود می‌شود.
امنیت	سرور در هر درخواست، نقش کاربر را بررسی می‌کند.	برای هر endpoint یک آزمون خودکار وجود دارد که نشان می‌دهد درخواست کاربر فاقد مجوز با خطای 403 رد می‌شود.
حریم خصوصی	رأی‌ها مخفی هستند.	اطلاعات «چه کسی رأی داده» و «به چه کسی رأی داده شده» در دو جدول جدا ذخیره می‌شوند و هیچ کاربری، حتی کارشناس، نمی‌تواند رأی یک فرد مشخص را ببیند.
دسترس‌پذیری	کاربرانی که نمی‌خواهند اپلیکیشن را نصب کنند، از طریق یک صفحهٔ وب متصل به همان API به سامانه دسترسی دارند.	همهٔ مراحل رأی‌گیری از طریق وب نیز انجام می‌شود و همان قواعد و محدودیت‌های اپ روی آن اعمال می‌شود.
قابلیت استفاده	نیاز به ورود دوباره تا حد ممکن کم است.	کاربری که دست‌کم هر ۳۰ روز یک بار از سامانه استفاده کند، نیازی به ورود دوباره ندارد.
قابلیت استفاده	رابط کاربری فارسی و ساده است.	همهٔ صفحات راست‌به‌چپ و با تاریخ شمسی هستند و ثبت درخواست یا رأی از صفحهٔ اصلی حداکثر در سه مرحله انجام می‌شود.
یکپارچگی داده	قواعد حساس را خود پایگاه داده تضمین می‌کند، نه فقط کد برنامه.	اگر دو رأی از یک نفر هم‌زمان ارسال شود، رأی دوم رد می‌شود و برای این حالت آزمون خودکار وجود دارد.
کارایی	سامانه در روز انتخابات نیز پاسخ‌گو می‌ماند.	با ۱۰۰ کاربر هم‌زمان، پاسخ ۹۵٪ درخواست‌ها کمتر از ۵۰۰ میلی‌ثانیه طول می‌کشد.
نگهداشت‌پذیری	سرور به‌سادگی و به یک شکل روی هر سیستمی راه‌اندازی می‌شود.	کل سرور (Django و PostgreSQL) با یک دستور `docker compose up` اجرا می‌شود.
سازگاری	اپلیکیشن روی بیشتر گوشی‌های اندرویدی اجرا می‌شود.	حداقل نسخهٔ پشتیبانی‌شده اندروید ۸ (API 26) است.

۶ الزامات عمومی و بیرونی محصول

۱.۶ مدیریت خطا

- **ورودی نامعتبر:** سرور همهٔ ورودی‌ها را اعتبارسنجی می‌کند و خطاها را در قالبی یکسان، شامل کد خطا و پیام فارسی، برمی‌گرداند. اپلیکیشن پیام را کنار همان فیلدی نشان می‌دهد که خطا داشته است.
- **قطع اینترنت:** اپلیکیشن آخرین داده‌های دریافت‌شده را از Room نمایش می‌دهد و به کاربر اطلاع می‌دهد که اتصال برقرار نیست. ثبت درخواست و ثبت رأی فقط در حالت متصل انجام می‌شوند تا وضعیت ناسازگار پیش نیاید.
- **خطا حین عملیات:** عملیات حساس داخل تراکنش اجرا می‌شوند؛ اگر عملیات شکست بخورد، هیچ بخشی از آن ذخیره نمی‌شود.

۲.۶ امکان نصب و اجرا

- **اپلیکیشن:** فایل APK از بخش Releases مخزن اندروید قابل دریافت و نصب است.
- **سرور:** کل سرور با یک دستور `docker compose up` اجرا می‌شود.
- **صفحهٔ وب و پنل کارشناسان:** هر دو از همان نشانی سرور در دسترس‌اند.

۳.۶ قابلیت ساخت از کد منبع

فایل `README` هر مخزن موارد زیر را مستند می‌کند:

- **پیش‌نیازها:** Docker و Docker Compose برای سرور؛ Android Studio و JDK 17 برای اپلیکیشن.
- **وابستگی‌ها:** فهرست کتابخانه‌ها در `requirements.txt` سرور و فایل‌های Gradle اپلیکیشن.
- **پیکربندی:** فایل نمونهٔ `.env.example` که متغیرهای محیطی لازم را نشان می‌دهد.
- **آماده‌سازی پایگاه داده:** migrationها هنگام بالا آمدن سرور به‌طور خودکار اجرا می‌شوند و ساخت حساب کارشناس با یک دستور مستندشده انجام می‌شود.
- **سرویس‌های خارجی:** تنظیمات سرویس اعلان، و نشانی آینه‌های داخلی برای Docker و Gradle در صورتی که دسترسی مستقیم ممکن نباشد.

۴.۶ تناسب محدودهٔ پروژه با منابع تیم

نسخهٔ قابل تحویل پروژه همان پنج قابلیت اصلی بخش ۲.۴ است و این حجم کار با سه عضو و تقسیم کار اندروید، سرور و یکپارچه‌سازی متناسب است. اهداف توسعه‌ای فقط پس از کامل شدن این پنج قابلیت و به ترتیب اولویت شروع می‌شوند؛ اگر زمان کم بیاید، ابتدا اهداف توسعه‌ای حذف می‌شوند، نه قابلیت‌های اصلی.

۷ فرایند کار تیم و برنامهٔ توسعه

۱.۷ انتخاب ابزارها و فناوری‌ها

جدول ۳: ابزارها و فناوری‌های انتخاب‌شده

حوزه	انتخاب	دلیل و مزیت	محدودیت
زبان اپلیکیشن	Kotlin	زبان رسمی اندروید؛ ایمنی در برابر null در خود زبان، که بخش بزرگی از خطاهای زمان اجرا را پیش از اجرا حذف می‌کند.	فقط برای اندروید؛ نسخهٔ iOS کد جداگانه لازم دارد.
رابط کاربری اپلیکیشن	Jetpack Compose	جعبه‌ابزار رسمی و جدید رابط کاربری اندروید با پشتیبانی خوب از راست‌به‌چپ.	یادگیری آن برای اعضای تازه‌کار زمان می‌برد.
ارتباط اپ با سرور	Retrofit و OkHttp	کتابخانهٔ استاندارد درخواست‌های HTTP در اندروید؛ تمدید خودکار توکن با `Authenticator` در OkHttp.	–
پایگاه دادهٔ اپلیکیشن	Room	کتابخانهٔ رسمی اندروید؛ بررسی درستی کوئری‌ها در زمان کامپایل و نمایش داده‌ها بدون اینترنت.	فقط نسخه‌ای محلی است؛ منبع اصلی داده سرور است.
زبان و فریم‌ورک سرور	Python، Django و Django REST Framework	محافظت‌های امنیتی پیش‌فرض، پنل مدیریت آماده و ساختار یکدست برای API.	کارایی پایین‌تر از زبانی مانند Go، که در مقیاس یک دانشگاه اثری ندارد.
احراز هویت	SimpleJWT	توکن کوتاه‌مدت و تمدید با چرخش؛ ورود ماندگار بدون کاهش امنیت.	عمر توکن‌ها باید با دقت تنظیم شود.
مستندات API	drf-spectacular و openapi-generator	تولید خودکار مستندات OpenAPI و ساخت کلاینت Kotlin از روی آن؛ اپ و سرور به یک قرارداد واحد متصل‌اند.	گاهی نوع دقیق یک فیلد باید دستی توضیح داده شود.
پایگاه دادهٔ سرور	PostgreSQL	قیدهای پیشرفته (ایندکس یکتای شرطی و قید عدم هم‌پوشانی) و migrationهای تراکنشی.	در هاست‌های اشتراکی کمتر رایج است؛ با Docker این محدودیت بی‌اثر است.
صفحات وب	Django Templates	بدون نیاز به زبان سوم؛ محافظت CSRF و escape خودکار خروجی.	تعامل‌پذیری محدود؛ در صورت نیاز HTMX یا یک صفحهٔ TypeScript اضافه می‌شود.
استقرار	Docker Compose	اجرای سرور با یک دستور و در محیطی یکسان روی سیستم همهٔ اعضا و سرور نهایی.	دسترسی به Docker Hub از ایران محدود است و ممکن است به آینه‌های داخلی نیاز باشد.

حوزه	انتخاب	دلیل و مزیت	محدودیت
طراحی رابط کاربری	Figma (اختیاری)	امکان طراحی و بازبینی صفحات پیش از پیاده‌سازی؛ در صورت انجام، مسئولیت آن با امیرمهدی عزیزی است.	به‌دلیل محدودیت زمانی ممکن است انجام نشود؛ در آن صورت صفحات مستقیماً با Compose طراحی می‌شوند.
کنترل نسخه و مدیریت وظایف	Git، GitHub و GitHub Projects	ثبت کد، وظایف و تصمیم‌ها در یک‌جا.	–
آزمون	چارچوب آزمون Django و JUnit	چارچوب‌های پیش‌فرض هر بستر، بدون نیاز به ابزار اضافه.	–
مستندات	LaTeX (XeLaTeX و xepersian)	حروف‌چینی دقیق فارسی و نگه‌داری متن مستندات در Git همانند کد.	یادگیری آن برای اعضای ناآشنا زمان می‌برد.

اجزای آماده: احراز هویت (Django و SimpleJWT)، پنل کارشناسان (Django Admin)، تولید مستندات API و کلاینت Kotlin، و ذخیره‌سازی محلی (Room).

اجزای پیاده‌سازی‌شده توسط تیم: مدل‌های داده و قیدها، گردش کار درخواست‌ها، سازوکار انتخابات با رأی مخفی، کنترل دسترسی بر اساس نقش، صفحهٔ وب انتخابات و همهٔ صفحات اپلیکیشن اندروید.

۱.۱.۷ گزینه‌های بررسی‌شده برای تصمیم‌های اصلی

فریم‌ورک سمت سرور. سه گزینه بررسی شد: Django (Python)، NestJS یا Next.js همراه با Prisma (TypeScript)، و Go. Django به چهار دلیل انتخاب شد:

۱. اعضای تیم با Python کار کرده‌اند، ولی هیچ‌کدام با TypeScript آشنا نیستند.

۲. محافظت‌های امنیتی Django به‌صورت پیش‌فرض فعال‌اند؛ این برای تیمی بدون متخصص امنیت که هدفش سامانه‌ای قابل اعتماد است، اهمیت اساسی دارد.

۳. پنل مدیریت آمادهٔ Django در فاز اول، رابط کارشناسان معاونت را تأمین می‌کند.

۴. قیدهای PostgreSQL که انتخابات و رویدادها به آن‌ها نیاز دارند، در Django به‌صورت داخلی پشتیبانی می‌شوند.

Next.js کنار گذاشته شد، زیرا اساساً فریم‌ورکی برای ساخت وب‌سایت است، در حالی که کلاینت اصلی این پروژه اپلیکیشن اندروید است. Go نیز کنار گذاشته شد، زیرا کد تکراری بیشتری لازم دارد و برتری آن در کارایی در مقیاس یک دانشگاه اثری ندارد.

پایگاه دادهٔ سرور. سه گزینه بررسی شد: PostgreSQL، MySQL و MongoDB. داده‌های فرنما ذاتاً رابطه‌ای‌اند (کاربر، تشکل، درخواست، انتخابات و رأی)، و پایگاه‌های NoSQL مانند MongoDB تضمین‌های لازم برای قیدها و تراکنش‌ها را به‌شکل داخلی ندارند. میان دو گزینهٔ رابطه‌ای، PostgreSQL انتخاب شد، زیرا سه امکان دارد که MySQL ندارد و هر سه مستقیماً به نیازهای پروژه مربوط‌اند:

- **ایندکس یکتای شرطی:** برای تضمین «فقط یک انتخابات باز برای هر تشکل».
- **قید عدم هم‌پوشانی:** برای جلوگیری از تداخل زمان و مکان رویدادها.

- **migrationهای تراکنشی:** برای امن‌تر کردن تغییر ساختار پایگاه داده در کار تیمی.

تنها مزیت MySQL، یعنی رایج بودن آن در هاست‌های اشتراکی، با استفاده از Docker بی‌اثر می‌شود.

۲.۱.۷ معماری سامانه

سطح کلان: کلاینت–سرور، API-محور. سامانه از یک سرور مرکزی تشکیل شده است که اپلیکیشن اندروید از طریق یک REST API نسخه‌دار با قرارداد OpenAPI به آن متصل می‌شود؛ صفحهٔ وب و پنل کارشناسان نیز روی همان سرور اجرا می‌شوند. همهٔ قواعد سامانه، یعنی مجوزها، گردش درخواست‌ها و قواعد انتخابات، فقط یک بار و در لایهٔ سرویس سرور تعریف می‌شوند و همهٔ کلاینت‌ها از همین لایه استفاده می‌کنند. سرور و پایگاه داده به‌صورت دو کانتینر با Docker Compose اجرا می‌شوند.

سرور: یکپارچهٔ ماژولار (Modular Monolith). سرور یک پروژهٔ Django است که بر اساس حوزه‌های کاری به چند ماژول تقسیم شده است: کاربران، تشکل‌ها، درخواست‌ها، انتخابات و اعلان‌ها. هر ماژول سه لایه دارد: نمایش، سرویس و داده. معماری ریزسرویس (Microservices) نیز بررسی شد، اما برای تیمی سه‌نفره در بازهٔ ۴۰ روزه، هزینهٔ استقرار و هماهنگی آن توجیه نداشت.

اپلیکیشن اندروید: MVVM. اپلیکیشن بر اساس معماری پیشنهادی گوگل ساخته می‌شود. صفحات Compose وضعیت خود را از ViewModel می‌گیرند و ViewModel داده را از Repository دریافت می‌کند. Repository داده را از سرور (Retrofit) می‌گیرد و در Room ذخیره می‌کند؛ به همین دلیل اپلیکیشن بدون اینترنت نیز آخرین داده‌ها را نشان می‌دهد. یکی از دلایل انتخاب این معماری آن است که مستندات رسمی اندروید بر پایهٔ آن نوشته شده‌اند و این، کار را برای عضوی که تازه اندروید را یاد می‌گیرد ساده‌تر می‌کند.

۲.۷ تعریف و توجیه نقش اعضای تیم

وظایف هر عضو در بخش ۱.۳ آمده است. در این بخش، ضرورت هر نقش، تناسب عضو با آن و ارتباط آن با سایر نقش‌ها توضیح داده می‌شود.

اندروید (زهرا میرزائی)

- **ضرورت نقش:** اپلیکیشن اندروید رابط اصلی دبیران، اعضای تشکل‌ها و رأی‌دهندگان است.
- **تناسب عضو با نقش:** با Python، Git و لینوکس کار کرده است. یادگیری Kotlin و Jetpack Compose بخشی از کار او در این پروژه است و امیرحسین عالیان، با تجربه‌ای که در برنامه‌نویسی اندروید دارد، پشتیبان این بخش است.
- **ارتباط با سایر نقش‌ها:** بر اساس قرارداد API کار می‌کند و از کلاینت تولیدشده از روی آن استفاده می‌کند.

سرور (امیرمهدی عزیزی)

- **ضرورت نقش:** همهٔ داده‌ها، قواعد و امنیت سامانه در سرور اجرا می‌شوند.
- **تناسب عضو با نقش:** با Python، Git و لینوکس کار کرده است. Django بر پایهٔ Python ساخته شده و سرور نیز با Docker روی محیط لینوکس اجرا می‌شود؛ بنابراین این نقش ادامهٔ مستقیم مهارت‌های فعلی اوست.
- **ارتباط با سایر نقش‌ها:** API را مطابق قرارداد پیاده‌سازی می‌کند و هر تغییر در قرارداد با هماهنگی نقش یکپارچه‌سازی انجام می‌شود. طراحی رابط کاربری در Figma، در صورت

انجام، با اوست.

یکپارچه‌سازی (امیرحسین عالیان)

- **ضرورت نقش:** اپ و سرور به‌طور مستقل توسعه داده می‌شوند؛ بدون نقشی مشخص برای حفظ یکپارچگی، ناسازگاری میان این دو بخش دیر کشف می‌شود.
- **تناسب عضو با نقش:** تجربهٔ برنامه‌نویسی اندروید دارد، با Python و Docker کار کرده و در لینوکس تسلط قابل قبولی دارد. او تنها عضوی است که با هر دو طرف پروژه، یعنی اپ و سرور، آشناست. رفع تعارض‌های ادغام کد در Git نیز بر عهدهٔ اوست.
- **ارتباط با سایر نقش‌ها:** میان دو نقش دیگر قرار دارد و در صورت نیاز در همهٔ بخش‌ها مشارکت می‌کند.

۳.۷ برنامهٔ زمان‌بندی و نقاط عطف

بازهٔ اجرای پروژه ۴۰ روز، از شنبه ۲۳ آبان تا یکشنبه ۲ دی ۱۴۰۵ است. دو هفتهٔ اول با میان‌ترم‌ها هم‌زمان است و سبک‌تر در نظر گرفته شده، و بیشتر کار پس از ۹ آذر انجام می‌شود.

جدول ۴: برنامهٔ زمان‌بندی هفتگی

هفته	فعالیت اصلی	مسئول اصلی	مسئول فرعی	معیار تکمیل
۱ (۲۳ تا ۲۹ آبان)	راه‌اندازی مخازن و Docker، یادگیری Django و Kotlin	امیرحسین عالیان	امیرمهدی عزیزی، زهرا میرزائی	سرور و اپ هر دو در حالت اولیه اجرا می‌شوند.
۲ (۳۰ آبان تا ۶ آذر)	احراز هویت و صفحهٔ ورود	امیرمهدی عزیزی	زهرا میرزائی	ورود کار می‌کند و توکن برمی‌گرداند.
۳ (۷ تا ۱۳ آذر)	مدیریت انجمن‌ها و پنل کارشناسان	زهرا میرزائی	امیرحسین عالیان	انجمن در پنل تعریف و در اپ نمایش داده می‌شود.
۴ (۱۴ تا ۲۰ آذر)	ثبت و پیگیری درخواست‌ها	امیرمهدی عزیزی	زهرا میرزائی	درخواست از اپ ثبت، در پنل تأیید و وضعیت آن در اپ دیده می‌شود.
۵ (۲۱ تا ۲۷ آذر)	انتخابات در اپ و صفحهٔ وب	امیرمهدی عزیزی	زهرا میرزائی، امیرحسین عالیان	رأی‌گیری از اپ و وب انجام می‌شود و رأی تکراری رد می‌شود.
۶ (۲۸ آذر تا ۲ دی)	اعلان‌ها، رفع اشکال و آمادگی تحویل	امیرحسین عالیان	امیرمهدی عزیزی، زهرا میرزائی	فایل APK و سند نهایی آماده‌اند.

مسئول اصلی هر هفته کسی است که کار آن هفته بیش از همه به او وابسته است. هفته‌های ۱ و ۶ کار زیرساخت و یکپارچه‌سازی دارند. در هفته‌های ۲، ۴ و ۵ اپ بدون آماده بودن API سرور پیش نمی‌رود. در هفتهٔ ۳ بیشترین حجم کار در اپ است (ورود ماندگار و صفحات انجمن‌ها). ترتیب هفته‌ها نیز از وابستگی قابلیت‌ها پیروی می‌کند: احراز هویت پیش از همه، انجمن‌ها پیش از درخواست‌ها (زیرا هر درخواست متعلق به یک انجمن است)، و انتخابات در پایان، زیرا پیچیده‌ترین قابلیت است و از ساختارهای قبلی استفاده می‌کند.

۴.۷ شناسایی و تحلیل ریسک‌ها

۱.۴.۷ ناآشنایی با فناوری‌های اصلی

- **شرح:** هیچ‌یک از اعضا با Django کار نکرده‌اند و زهرا میرزائی تجربهٔ برنامه‌نویسی اندروید ندارد. این ناآشنایی دو پیامد احتمالی دارد: تأخیر در پیشرفت کار، و ضعف امنیتی؛ برای نمونه، endpointی که نقش کاربر را بررسی نمی‌کند یا تنظیم نادرست احراز هویت، که می‌تواند به افشای اطلاعات یا دست‌کاری در انتخابات منجر شود.
- **علت احتمالی:** تجربهٔ محدود تیم در فناوری‌های انتخاب‌شده.
- **احتمال وقوع:** زیاد برای تأخیر؛ متوسط برای ضعف امنیتی.
- **میزان اثرگذاری:** متوسط برای تأخیر؛ زیاد برای ضعف امنیتی، زیرا اعتماد به انتخابات، یکی از اهداف اصلی محصول، را از بین می‌برد.
- **اقدام پیشگیرانه:** اختصاص هفته‌های ۱ و ۲ به یادگیری و کارهای ساده و پشتیبانی امیرحسین عالیان از بخش اپ؛ استفاده از محافظت‌های پیش‌فرض Django و کتابخانه‌های جاافتاده مانند SimpleJWT به‌جای نوشتن سازوکار امنیتی از صفر؛ تضمین قواعد حساس در خود پایگاه داده؛ آزمون خودکار دسترسی غیرمجاز برای هر endpoint؛ بازبینی هر تغییر کد از طریق Pull Request؛ و اجرای `check --deploy` پیش از هر تحویل.
- **برنامهٔ مقابله:** در صورت تأخیر، قابلیت‌ها با امکانات آمادهٔ Django ساده‌تر پیاده‌سازی می‌شوند. در صورت کشف حفرهٔ امنیتی، حفره فوراً بسته می‌شود و همهٔ توکن‌های فعال باطل می‌شوند تا کاربران دوباره وارد شوند؛ اگر سلامت یک انتخابات زیر سؤال برود، آن انتخابات باطل و تکرار می‌شود.

۲.۴.۷ محدودیت دسترسی از ایران به سرویس‌های خارجی

- **شرح:** دریافت ایمیج‌های Docker، وابستگی‌های Gradle و استفاده از سرویس اعلان گوگل ممکن است مسدود یا ناپایدار باشد.
- **علت احتمالی:** تحریم‌ها و محدودیت‌های اینترنت.
- **احتمال وقوع:** زیاد.
- **میزان اثرگذاری:** زیاد؛ ساخت پروژه یا ارسال اعلان متوقف می‌شود.
- **اقدام پیشگیرانه:** شناسایی و تنظیم آینه‌های داخلی Docker و Gradle در هفتهٔ اول.
- **برنامهٔ مقابله:** اگر هیچ سرویس اعلانی در دسترس نباشد، اپلیکیشن هنگام باز شدن، اعلان‌های جدید را از سرور دریافت می‌کند.

۳.۴.۷ کمبود زمان

- **شرح:** پنج قابلیت اصلی باید در ۴۰ روز تکمیل شوند و دو هفتهٔ اول این بازه با میان‌ترم‌ها هم‌زمان است.
- **علت احتمالی:** بازهٔ زمانی فشرده و هم‌زمانی با پروژه‌ها و امتحان‌های درس‌های دیگر.
- **احتمال وقوع:** متوسط.
- **میزان اثرگذاری:** زیاد.
- **اقدام پیشگیرانه:** بررسی هفتگی پیشرفت در GitHub Projects و پیاده‌سازی قابلیت‌ها به ترتیب وابستگی.

- **برنامهٔ مقابله:** جابه‌جایی وظایف میان اعضا؛ اهداف توسعه‌ای پیش از قابلیت‌های اصلی حذف می‌شوند.

۴.۴.۷ ناسازگاری اپ و سرور

- **شرح:** ممکن است یک endpoint در سرور تغییر کند، بدون اینکه بخش اندروید از آن باخبر شود.
- **علت احتمالی:** توسعهٔ همزمان اپ و سرور توسط دو نفر.
- **احتمال وقوع:** متوسط.
- **میزان اثرگذاری:** متوسط؛ خطاها دیر کشف می‌شوند.
- **اقدام پیشگیرانه:** قرارداد OpenAPI به‌عنوان مرجع واحد و تولید خودکار کلاینت اپ از روی آن؛ هر تغییر قرارداد از طریق Pull Request انجام می‌شود.
- **برنامهٔ مقابله:** کلاینت دوباره تولید می‌شود؛ در این حالت ناسازگاری‌ها به خطای کامپایل تبدیل و همان روز رفع می‌شوند.

۵.۷ برنامهٔ دریافت بازخورد

جدول ۵: مراحل دریافت بازخورد

مرحله	از چه کسانی	چه چیزی ارائه می‌شود
پس از تحویل پروپوزال	استاد درس	سند پروپوزال
پایان هفتهٔ ۳ (۱۳ آذر)	دبیران دو یا سه انجمن	نسخهٔ آزمایشی APK با صفحات ورود و انجمن‌ها، یا طرح Figma در صورت آماده بودن
پایان هفتهٔ ۵ (۲۷ آذر)	دبیران انجمن‌ها	نسخهٔ آزمایشی APK و صفحهٔ وب، همراه با یک انتخابات آزمایشی میان خود دبیران
آغاز هفتهٔ ۶ (۲۸ آذر)	کارشناسان معاونت	پنل کارشناسان و سامانهٔ کامل، برای بررسی گردش درخواست‌ها و انتخابات

- **ثبت بازخوردها:** هر بازخورد به‌صورت یک Issue با برچسب `feedback` در GitHub ثبت می‌شود و منبع و مرحلهٔ دریافت آن نیز کنارش نوشته می‌شود.
- **اولویت‌بندی:** هر بازخورد بر اساس دو معیار سنجیده می‌شود: میزان اثر بر قابلیت‌های اصلی و هزینهٔ پیاده‌سازی، و بر این اساس در یکی از سه دستهٔ «ضروری»، «مفید» و «برای بعد» قرار می‌گیرد.
- **تصمیم‌گیری:** در جلسهٔ هفتگی تیم در تلگرام دربارهٔ بازخوردها تصمیم گرفته می‌شود. پیشنهادهایی که با محدودهٔ پروژه سازگارند و در زمان باقی‌مانده جا می‌شوند اعمال می‌شوند و بقیه به فهرست اهداف توسعه‌ای اضافه می‌شوند.

دریافت بازخورد از دانشجویان عمومی در برنامهٔ فعلی پروژه قرار ندارد و ممکن است در مراحل آینده انجام شود.